NUNNAGOPPALA HARI PRASAD

Vijayawada, India | +91 9704813856 | hariprasad97048@gmail.com | LinkedIn | GitHub

PROFESSIONAL SUMMARY

AI Engineer with 2+ years of experience building production-grade agentic AI systems in both Java and Python. Designed and shipped a dual-implementation (Spring Boot + LangGraph4j, and Python + LangGraph) agentic test-maintenance engine that autonomously triages and self-heals nightly mobile-web test failures, cutting manual QA triage effort by ~80-87% and LLM inference cost by 50-70% through a 3-tier cost-disciplined classification architecture. Strong foundation in RAG, multi-agent orchestration, CI/CD integration, and AWS.

TECHNICAL SKILLS

Languages: Java, Python, SQL, JavaScript

AI/ML & Agentic Systems: Agentic AI, LangGraph, LangGraph4j, RAG, Multi-Agent Orchestration, Model Context Protocol (MCP), Reflexion (Self-Correction Loops), Prompt Engineering, LLM Orchestration, Google ADK, LangChain, Vector Databases

Frameworks & Platforms: Spring AI, Spring Boot 3, Docker, AWS, Generative AI

MCP Architecture: Server-hosted MCP (JIRA MCP, GitHub MCP, BrowserStack MCP), IDE-hosted/Local MCP (WindSurf Cascade)

QA Automation: Appium, Selenium, REST Assured, BrowserStack, CI/CD Automation, Rule Engines

Tools: Git, GitLab, JIRA, Zephyr, GitHub

Continuous Learning: CrewAI, AutoGen, LangSmith, RAGAS / LLM-as-Judge evaluation

PROFESSIONAL EXPERIENCE

Assistant System Engineer — 2024 – Present

Tata Consultancy Services (TCS)

- Designed and built two separate agentic test-maintenance frameworks: Java/Spring Boot/LangGraph4j (server-hosted MCP) and Python LangGraph (IDE-hosted/local MCP via WindSurf Cascade).
- Delivered ~80-87% reduction in nightly QE triage effort and ~50-70% LLM cost reduction.
- Contributed to Algolia search implementation and MongoDB query optimization on a separate client-facing application.
- Architected a supervisor-driven multi-agent workflow consisting of Evidence, Triage, Fix, Bug Review, Test Writer, Escalation, and Feedback agents.
- Built the orchestration layer using LangGraph4j (Java) and LangGraph (Python).
- Designed Retrieval-Augmented Generation (RAG) using AWS Bedrock Knowledge Bases for execution history, functional documentation, UI semantic knowledge, and JIRA history.
- Implemented semantic search and vector retrieval to improve AI decision-making accuracy.
- Built automated BrowserStack MCP integration to collect screenshots, logs, videos, and DOM snapshots.
- Integrated GitHub MCP for automated Pull Request generation.
- Integrated JIRA MCP for automated defect creation and duplicate detection.
- Designed event-driven cloud architecture using AWS Lambda, EventBridge, Amazon S3, DynamoDB, Docker, AWS AgentCore, ECR, CloudWatch, and Secrets Manager.
- Implemented intelligent failure classification using rule-based filtering, semantic retrieval, and LLM reasoning.
- Designed Reflexion-based self-correction workflows to improve autonomous decision accuracy.

- Applied cost optimization techniques including semantic caching, prompt caching, model routing, parallel execution, and cluster-based failure analysis.
- Reduced manual regression maintenance effort by approximately 90% while significantly lowering AI inference costs.
- Designed reusable Spring Boot automation components for enterprise applications.
- Reduced project setup effort by 50% through reusable framework design.
- Improved backend memory utilization by 60% using optimized application architecture.
- Integrated automated CI/CD pipelines for build, testing, and deployment.
- Designed backend indexing workflows using Spring Batch and Spring MVC.
- Developed optimized MongoDB queries for large-scale search indexing.
- Built data transformation pipelines for Algolia indexing.
- Improved search accuracy and overall user experience through optimized backend processing.

PROJECT EXPERIENCE

Agentic Test-Maintenance Engine

Java (Spring Boot + LangGraph4j) & Python (LangGraph) | 2024 – 2025

Per-failure triage: 15-25 min → 1-2 min (~90%) | **Locator/wait fix:** 20-40 min → 3-5 min (~85%) | **Bug railed with evidence:** 15 min → 1 min (~93%)

- Architected an end-to-end agentic system across a 10-node stateful LangGraph pipeline (ingest, cluster, evidence-gathering, classify, validate, action, report) with a Reflexion self-correction loop and checkpointing for durable, resumable runs.
- Engineered a 3-tier failure classifier (deterministic rules → cross-run history cache → LLM only on the ambiguous remainder), plus per-task model routing, SHA-256 response caching, and a budget kill-switch for cost discipline.
- Automated locator/wait-sync remediation: gathering DOM/screenshots/BrowserStack evidence via BrowserStack MCP, deep-verifying fixes against the captured DOM, and opening draft PRs via GitHub MCP — with forbidden-path guardrails blocking edits to CI config, hooks, or driver/session code.
- Built two distinct MCP architectures — server-hosted (JIRA/GitHub/BrowserStack MCP) for Java/LangGraph4j enabling unattended CI, and IDE-hosted/local via WindSurf Cascade for Python/LangGraph enabling secure dev-time orchestration — unified by a JSON Agent Action Contract; every PR and JIRA/Zephyr draft requires human approval, neither agent ever auto-merges.

EDUCATION

Bachelor of Technology in Computer Science & Engineering
2023 | CGPA: 7.9 / 10

12th Grade
98.2%

10th Grade
100%